

Charging Hub Berlin – Team 05 Report

Team Members:

1. Ravi Pandit (110469)
2. Sunbla Khan (110344)
3. Krishnakumar Mudaliar (110932)
4. Ebna Sina

GitHub Repository:

<https://github.com/rvavii/ChargeHubBerlin>

1. Introduction and Project Objectives

- *ChargeHubBerlin* is an advanced software engineering project aimed at managing and improving urban electric vehicle (EV) charging infrastructure.
- Due to the increasing adoption of electric mobility in cities like Berlin, reliability, transparency, and maintainability of charging stations are critical.
- The project transforms raw and heterogeneous datasets into a structured and scalable backend system following Domain-Driven Design (DDD) principles.
- A key objective is to accurately model real-world charging infrastructure processes through a robust domain model.
- The architecture clearly separates domain logic, application services, and infrastructure components to ensure maintainability and extensibility.
- The project emphasizes Test-Driven Development (TDD), where core business logic is developed incrementally using automated tests.
- TDD ensures high code quality, regression safety, and alignment with modern software engineering best practices.
- The system is designed to be scalable and extensible, enabling future features such as predictive maintenance or data-driven analytics without major architectural changes.

1.1 Identified Use Cases

- Two core use cases were identified through **domain exploration** and **collaborative modeling sessions** using the Miro board.
- These use cases were prioritized based on their relevance to **end users, infrastructure operators, and system administrators**.

Use Case 1: Station Rating and Review Management

- Allows EV users to submit **star ratings** and **textual reviews** after using a charging station.
- Enhances **transparency** for future users when selecting charging stations.
- Provides operators with **actionable insights** into service quality and station performance.
- Ratings and reviews are modeled within a **dedicated domain aggregate**.
- Domain rules ensure **data validation**, consistency, and controlled aggregation of feedback.
- Objective: Establish a structured feedback loop enabling **data-driven service improvements** and **service optimization**.

Use Case 2: Malfunction Reporting and Incident Resolution

- Enables users to report **technical malfunctions** at charging stations.
- Each report follows a defined lifecycle (e.g., **reported, in progress, resolved**).
- Supports **real-time status updates** for all stakeholders.
- Facilitates **coordinated maintenance workflows** between administrators and operators.
- Objective: Reduce downtime, improve system reliability, and increase **user trust** through transparent and structured incident management.

1.3 Technology Stack

The ChargeHubBerlin project leverages a modern and integrated technology stack to support scalable backend development, domain modeling, testing, and collaborative teamwork.

- **Python** – Core programming language for backend logic and data processing
- **Visual Studio Code** – Primary IDE for development and testing
- **Streamlit** – Lightweight frontend for demonstrating application features
- **SQLite** – Relational database for structured data storage
- **DB Browser for SQLite** – Database inspection and validation tool
- **GitHub & GitLab** – Version control, collaboration, and CI support
- **draw.io & Miro** – Architecture, domain modeling, and sequence diagrams
- **AI Tools (ChatGPT, Perplexity)** – Test generation, refactoring, and documentation support
- **Google Meet** – Team communication and coordination

2. Project Development Documentation:

2.1 Domain Modelling and Event Structure

/core - Core application logic

/datasets - Data files for Berlin districts and postal codes

/images - Image assets

/src - Main source code

/auth - Authentication module

/malfunction - Malfunction tracking bounded context

/application - Application services

/domain - Domain model (DDD structure)

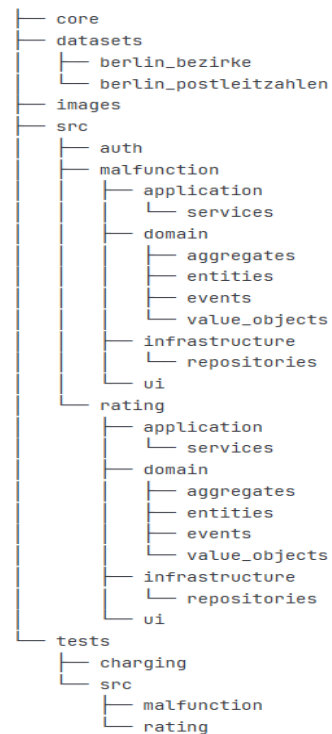
/infrastructure - Infrastructure layer (repositories, etc.)

/ui - User interface components

/rating - Rating system bounded context

(Same DDD structure as malfunction)

/tests - Test suite



2.2 Test-Driven Development (TDD) Documentation

2.1 TDD Approach

Development Workflow

The team used **VS Code Live Share** for real-time collaborative development in a shared workspace. Each team member took ownership of specific bounded contexts (malfunction and rating modules), following the TDD cycle: write tests first, implement code, then refactor.

Coordination

- Team worked collaboratively in one workspace using Live Share
- Each member handled specific modules while sharing ideas and code reviews in real-time
- Test-first approach for domain logic components
- Iterative development with immediate feedback

2.2 TDD Adherence

Strict TDD (Tests First):

- Domain value objects (Email, Name, StationLabel, Stars, etc.)
- Domain aggregates (IncidentAggregate, RatingAggregate)
- Application services (MalfunctionService, RatingService)
- Business rule validation

Partial TDD (Implementation First):

- DDD structure setup (first-time learning)
- Streamlit UI components (exploratory implementation)
- Main orchestration layer
- Infrastructure repositories

2.3 Test Coverage (~80% Target)

Coverage by Component

Component	Coverage	Test Count
Value Objects	~100%	16 tests
Aggregates	~90%	9 tests
Services	~85%	5 tests
Repositories	~70%	Indirect via services
UI	~20%	Manual testing

Total: 30+ test functions

Coverage Strategy

- **Pytest fixtures** for reusable test database setup
- **Temporary in-memory SQLite** for test isolation
- **Parametrized tests** for multiple scenarios

- **Error path testing** for exception handling
- **Integration tests** covering service → domain → repository layers

2.4 Backend Architecture

DDD Layer Structure

UI Layer (Streamlit) → Application Services → Domain
(Aggregates/Entities/VOs) → Infrastructure (Repositories/SQLite)

Core Components

Domain Layer:

- **Aggregates:** IncidentAggregate, RatingAggregate (enforce business rules, emit events)
- **Entities:** Incident, Rating
- **Value Objects:** Email, Name, StationLabel, ProblemDescription, Stars (immutable, self-validating)
- **Events:** MalfunctionReportOpened, PointsAwardedToReporter, ReportValidatedAndSubmitted

Application Layer:

- **MalfunctionService:** Submit, validate, and resolve malfunction reports
- **RatingService:** Create ratings and calculate averages

Infrastructure Layer:

- **Repositories:** IncidentRepository, SqliteRatingRepository
- **Database:** SQLite (malfunction.db, ratings.db)
- **External Services:** RealStationLookup (CSV-based Berlin station validation)

Data Model

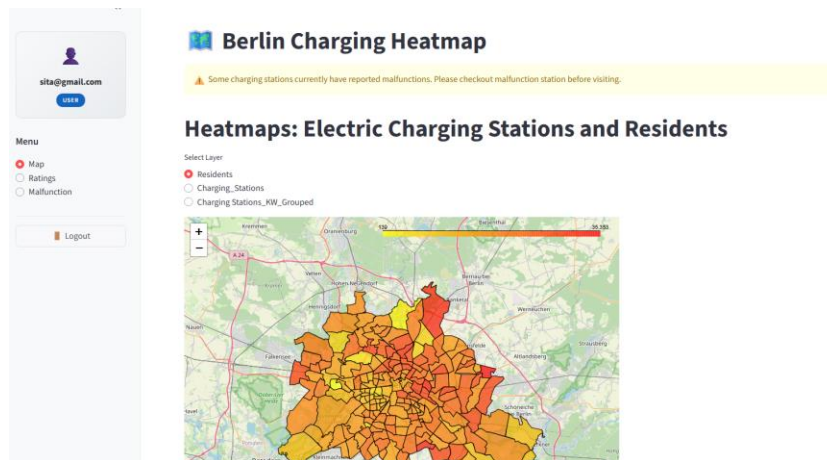
- **Production:** SQLite databases (malfunction.db, ratings.db)
- **Testing:** In-memory SQLite (:memory:) for complete isolation

2.5 UI Components

Application Structure

- **Main Navigation:** Three pages (Map, Ratings, Malfunction) via Streamlit sidebar
- **Global Warning:** Alert banner when open malfunctions exist
- **Data Loading:** Charging station CSV loaded once, passed to UI modules

User Flows



Ratings Page

Rate Station Section:

1. Select station from dropdown (Berlin stations only)
2. Enter name, email, star rating (1-5), optional review
3. Submit → Validation → Success with average rating displayed

View Ratings Section: *(Planned)*

- Display ratings for selected station
- Show average and review count

Malfunction Page

Report Section:

1. Enter name, email, select station, describe problem
2. Submit → Report created with PENDING status
3. Success confirmation

Malfunction List Section:

- Admin actions: Validate (awards 10 points), Resolve (add solution)
- Real-time updates

Technical Details

Key Features:

- Dynamic database path resolution (handles Streamlit working directory)
- Domain exception handling → user-friendly error messages
- Stateless operations (fresh DB connections per render)
- Two-layer validation (UI input + value objects)

Planned Optimizations

- Implement "View Station Ratings" display
- Add admin authentication
- Display reporter points in malfunction list
- Pagination for large lists
- Test coverage screenshots

3. Technical Challenges During Development Phase

Challenge 1: Domain-Driven Design (DDD) Implementation

DDD methodology was the first attempt by our team, so the workflow assembly was one challenge. The team had to learn and adapt to:

- Structuring the project with aggregates, entities, value objects, and domain events
- Separating concerns across application, domain, infrastructure, and UI layers
- Implementing the repository pattern correctly

Challenge 2: Test Isolation with Temporary Databases

Running the test files using temporary databases without involving the main SQLite database required careful configuration. The solution involved:

- Using pytest fixtures with temporary database instances
- Ensuring proper setup and teardown for each test
- Configuring conftest.py files to manage test database lifecycle

4. Project Completion

Technical Innovations

1. Domain-Driven Design (DDD) Implementation

- a. We structured the backend around **bounded contexts** and **aggregates**, clearly separating domain logic, application services, and infrastructure.

- b. This modular approach improved maintainability, ensured consistent business rules, and simplified the addition of new features like predictive maintenance or analytics.
- 2. Test-Driven Development (TDD) Adoption**
 - a. Core logic was developed following TDD principles, ensuring **high test coverage (~80%)** and minimizing regressions.
 - b. Automated tests also facilitated safer refactoring and increased confidence when integrating new features.
- 3. AI-Assisted Development**
 - a. We leveraged tools like **ChatGPT** and **Perplexity** for generating test templates, code refactoring suggestions, and documentation support.
 - b. This significantly reduced development time and enhanced code quality, especially for repetitive or boilerplate tasks.
- 4. Real-Time Incident Handling Logic**
 - a. The **malfunction reporting system** introduced a structured workflow with defined states (reported → in progress → resolved).
 - b. This innovation improved system reliability and demonstrated effective event-driven design in practice.
- 5. Lightweight Frontend Integration**
 - a. Using **Streamlit**, we quickly developed an interactive UI to visualize station data, user ratings, and malfunction reports.
 - b. This approach allowed rapid prototyping while keeping backend logic fully decoupled.

Lessons Learned

- **Importance of Clear Domain Modeling**

A well-defined domain model simplifies implementation, testing, and future extensions. Early investment in DDD pays off in long-term maintainability.
- **Collaboration & Coordination**

Version control (GitHub, GitLab) and regular meetings (Google Meet) were essential for synchronizing TDD practices across the team.
- **AI as a Productivity Tool**

Integrating AI tools can significantly accelerate development, but human oversight remains critical for validating logic and domain rules.
- **Iterative Approach**

Developing in small, testable increments enabled smoother integration and

reduced errors during deployment, reinforcing the value of incremental, TDD-based development.